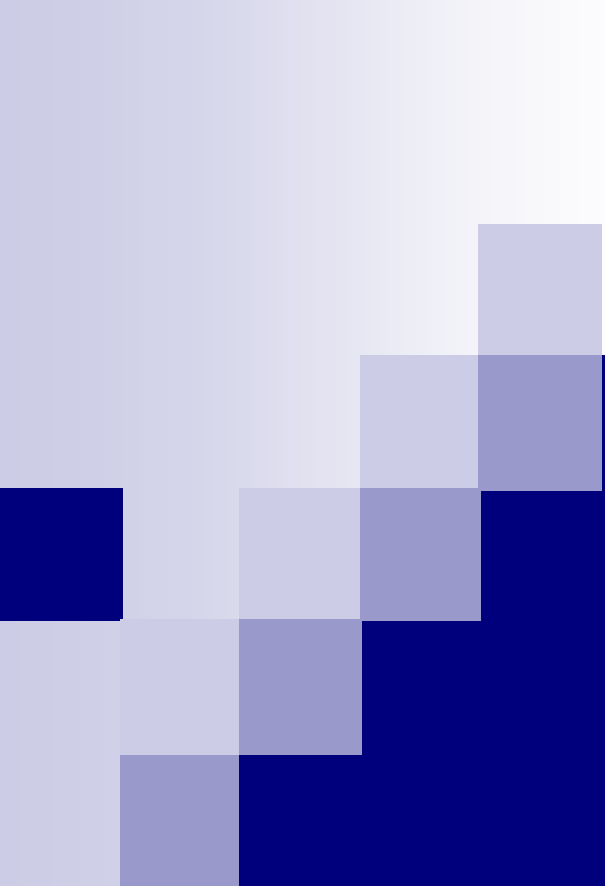




Software Design and SQA



Lecture # 10

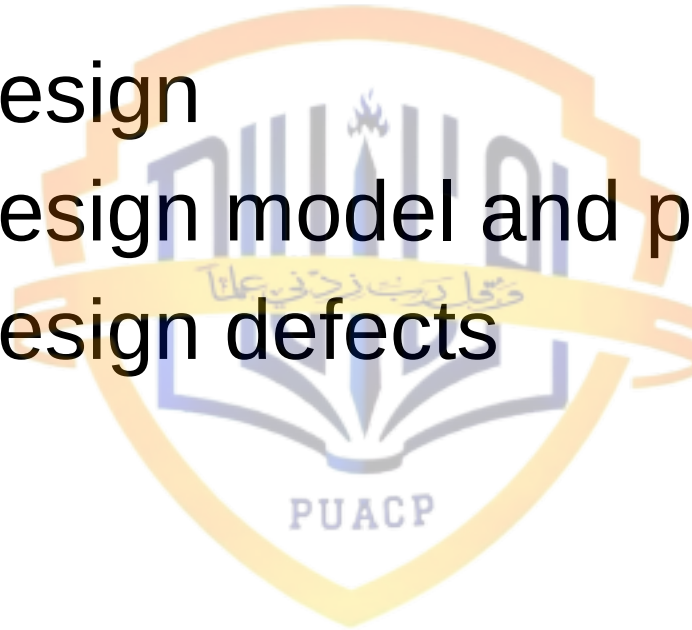
Recap

- Software requirements and software quality assurance



Today's Lecture

- Software design
- Software design model and process
- Software design defects



Design?

- Synonyms: plan, arrangement, lay out, map, scheme
- Antonyms: accident, fluke, chance, guess
- Design is an activity of creating a solution that satisfies a specific goal or need
- Design is the backbone of all products and services
- Considered an artistic and heuristic activity

Software Design - 1

- Software design is an artifact that represents a solution, showing its main features and behavior, and is the basis for implementing a program or collection of programs

Software Design - 2

- Design is a meaningful representation of something that is to be built. It can be traced to a customer's requirements and at the same time assessed for quality against a set of pre-defined criteria of “good” design

Software Design - 3

- Software design is different from other forms of design because it is not constrained by physical objects, structures, or laws
- As a result, software design tends to be much more complex than other forms of design because it is conceptually unbounded, whereas the capabilities of the human mind are bounded

Software Design - 4

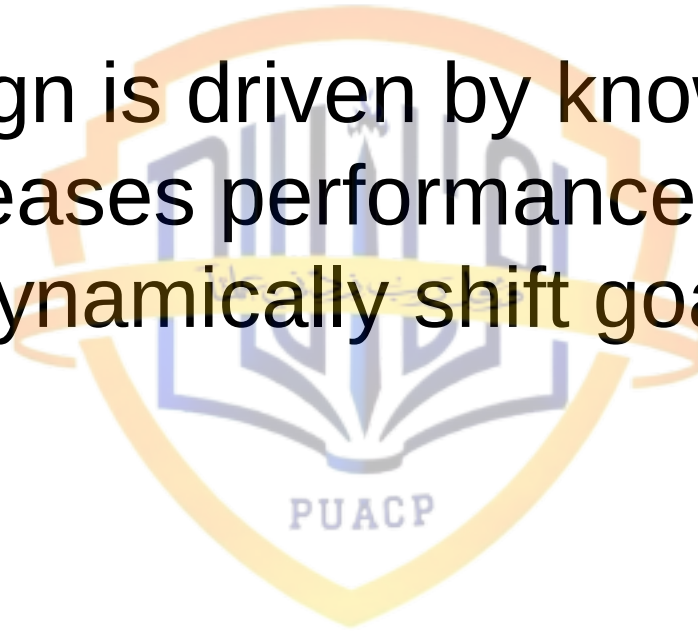
- Tasks are generally ill-defined and suffer from incomplete and inaccurate specifications
- There is seldom a predefined solution, although many solutions tend to satisfy the defined task

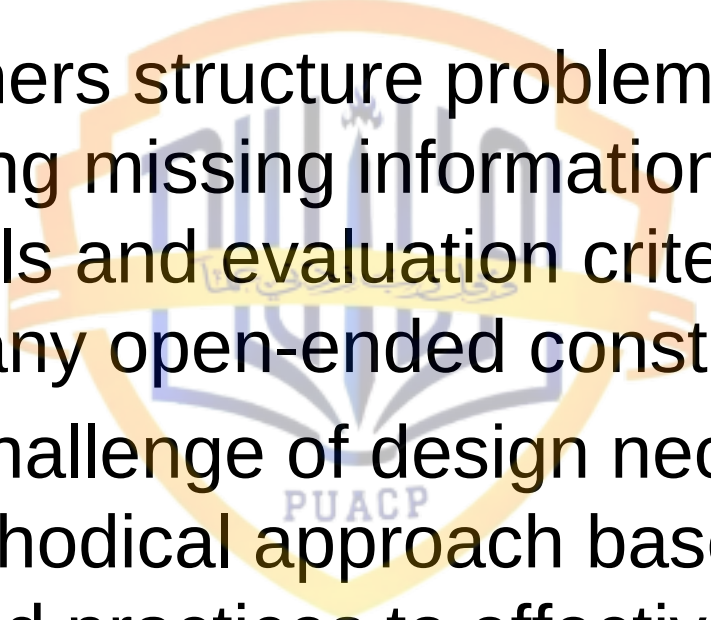
Software Design - 5

- Viable solutions usually require broad and interdisciplinary knowledge and skill, some of which is based on rapidly changing technology
- Solutions often involve many components that have numerous interactions

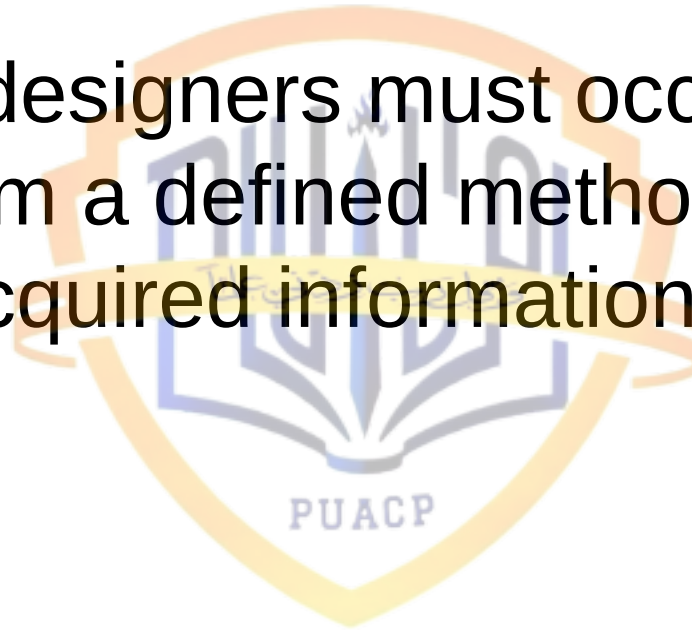
- 
- Expert designers use a mostly breadth-first approach because it allows them to mentally simulate the execution of an evolving system to detect unwanted interaction, inconsistencies, weaknesses, and incompleteness of their designs

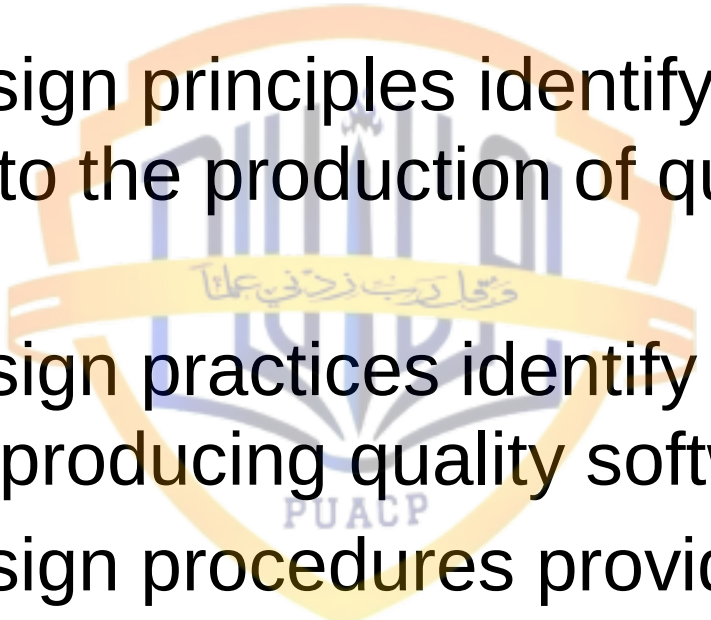
- 
- Thus, design is driven by known solutions, which increases performance by allowing a user to dynamically shift goals and activities



- 
- 
- Good designers structure problem formulations by discovering missing information, such as problem goals and evaluation criteria, and resolving many open-ended constraints
 - Hence the challenge of design necessitates the use of a methodical approach based on key principles and practices to effectively and efficiently produce high quality software designs

- 
- However, designers must occasionally deviate from a defined method in response to newly acquired information of insights



- 
- 
- Software design principles identify strategic approaches to the production of quality software designs
 - Software design practices identify tactical methods for producing quality software designs
 - Software design procedures provide an organizational framework for designing software

An Important Point

- Try to associate quality attributes with every aspect of software design

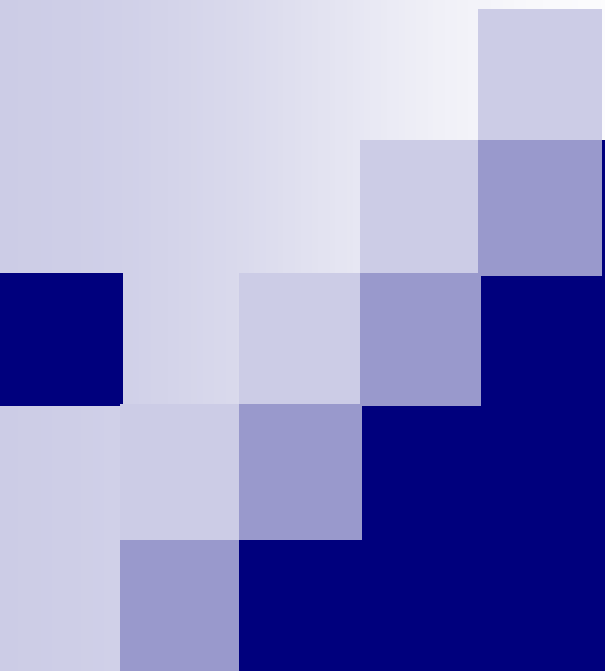


Design and Quality

- Design is the place where quality is fostered in software engineering
- Design provides us with representation of software which can be assessed for quality
- Design is the only way that we can accurately translate a customer's requirements into a finished software product or system

Without Software Design

- We risk building an unstable system
 - one that will fail when small changes are made
 - one that may be difficult to test
 - one whose quality cannot be assessed until late in the software process
 - one that will be of no or very little use for similar projects (not reusable)



Design Process and Model



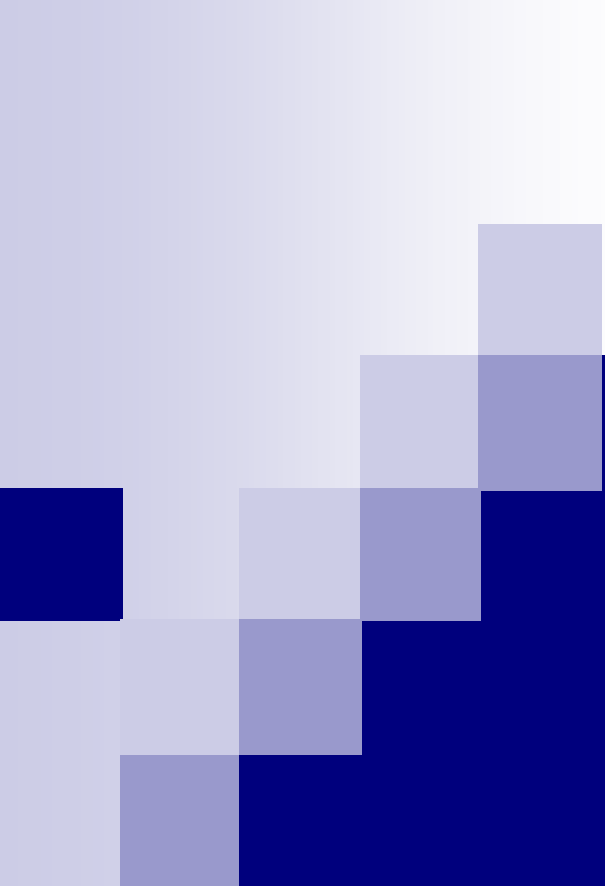
Software design is both a process and a model

Design Process

- It is a sequence of steps that enables a designer to describe all aspects of the software to be built
- During the design process, the quality of the evolving design is assessed with a series of formal technical reviews or design walkthroughs
- Needs creative skills, past experience, sense of what makes “good” software, and an overall commitment to quality

Design Model

- Equivalent to an architect's plan for a house
- Represents the totality of the thing to be built
- Provides a variety of different views of the computer software



Design Defects



PUACP

Design Defects - 1

- Defects introduced during preliminary design phase are usually not discovered until integration testing, which is too late in most cases
- Defects introduced during detailed design phase are usually discovered during unit testing

Design Defects - 2

- All four categories of defects are found in design models
 - Errors of commission
 - Errors of omission
 - Errors of clarity and ambiguity
 - Errors of speed and capacity

Design Defects - 3

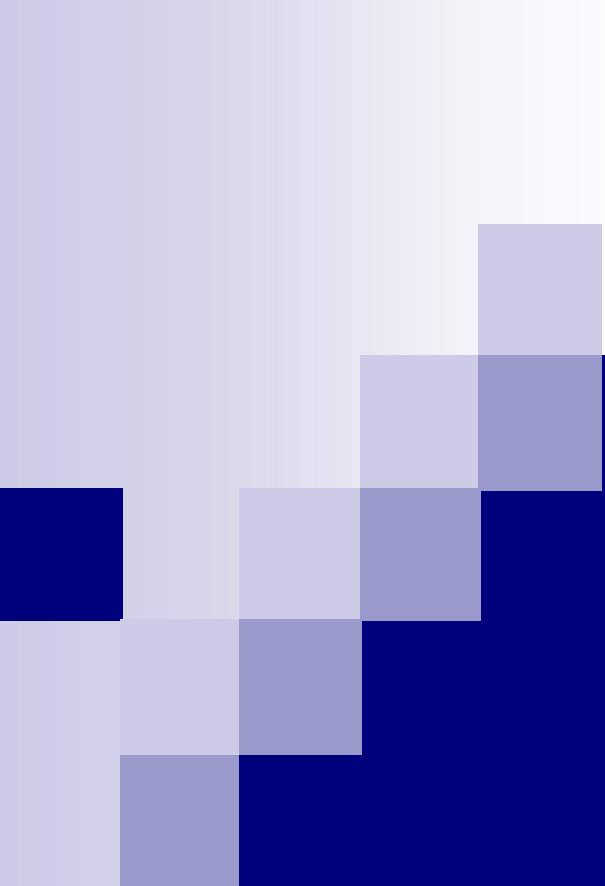
- Most common defects are errors of omission, followed by errors of commission
- Errors of clarity and ambiguity are also common, and many performance related problems originate in design process also

Design Defects - 4

- Overall design ranks next to requirements as a source of very troublesome and expensive errors
- A combination of defect prevention and defect removal is needed for dealing with design defects

Design Defects - 5

- Formal design inspections are one of the most powerful and successful software quality approaches of all times
- Software professionals should incorporate inspections in their software development process



Following slide to be
inserted

Defects in Fundamental
Design Topics

Defects in Fundamental Design Topics

- Functions performed
- Function installation, invocation, control, and termination
- Data elements
- Data relationships
- Structure of the application
- Sequences or concurrency of execution
- Interfaces



Functions Performed

- Errors in descriptions of functions the application will perform, are often errors of omission
- Often omitted functions are those which, are implied functions, rather than the explicitly demanded functions

Function Installation, Invocation, Control, and Termination

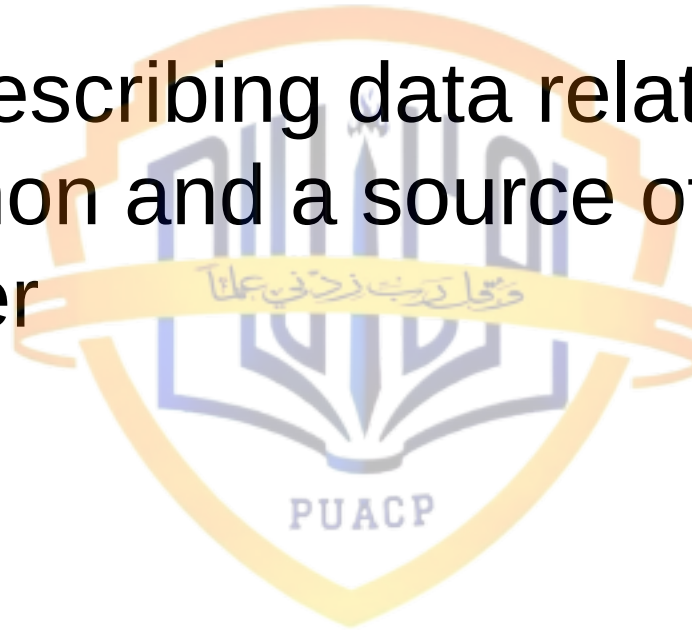
- Defects in information on how to start-up a feature, control its behavior, and safely turn off a feature when finished are common in commercial and in-house software applications
- Fifty percent of the problems reported to commercial software vendors are of this class

Data Elements

- Errors in describing the data used by the application are a major source of problems downstream during coding and testing
- A minor example of errors due to inadequate design of data elements can be seen in many programs that record addresses and telephone numbers
- Often insufficient space is reserved for names, etc.

Data Relationships

- Errors in describing data relationships are very common and a source of much trouble later



Structure of the Application

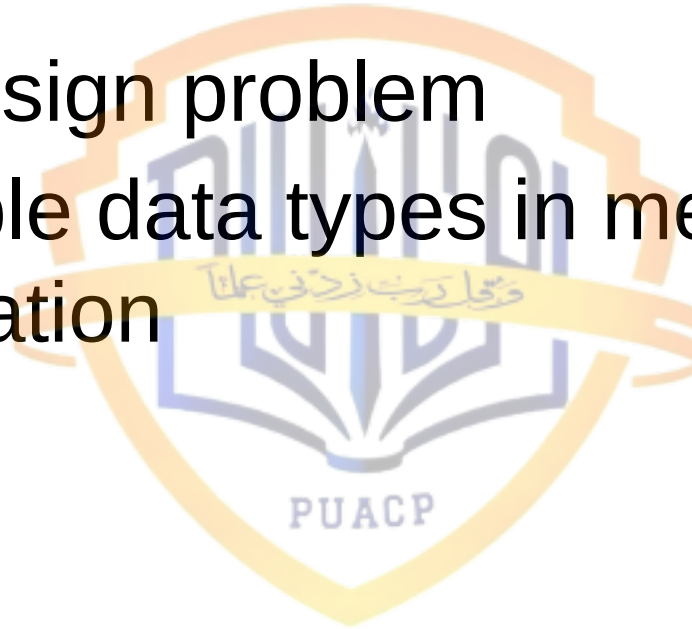
- Complex software structures with convoluted control flow tend to have higher error rates
- Poor structural design is fairly common, and is often due to haste or poor training and preparation
- Tools can measure cyclomatic and essential complexity
- Prevention is often better than attempting to simplify an already complex software structure

Sequences or Concurrency of Execution

- Many errors of speed and capacity have their origin in failing to design for optimum performance
- Performance errors are a result of complex control flow, excessive branching, or too many sequential processing (use parallel processing)
- Minimize I/O operations

Interfaces

- Chronic design problem
- Incompatible data types in message communication



Errors in Eight Secondary Design Topics

- Security
- Reliability
- Maintainability
- Performance
- Human factors
- Hardware dependencies
- Software dependencies
- Packaging



Addressing Design Problems

- Continuously evaluate your design model and design process
- Use design inspections or formal technical reviews, which have proven to be the most valuable mechanism to improve quality of software ever, and especially for design
- Develop software design by following design principles and guidelines

References

- Software Quality: Analysis and Guidelines for Success by Capers Jones
- Software Engineering: A Practitioner's Approach by Roger Pressman (Chapter 13)
- Software Engineering Quality Practices by Ronald K. Kandt